并行程序设计与算法实验
实验一：基于 MPI 的并行矩阵乘法

姓名：元朗曦　学号：23336294

2026 年 4 月 9 日

1　实验目的

本实验的目标是使用 MPI 实现通用矩阵乘法的并行计算，并分析不同进程数量和不同矩阵规模下的性能变化。通过本实验希望达到以下目的：

1. 熟悉 MPI 的基本编程模型，以及主从式并行计算的组织方式。
2. 掌握矩阵乘法的分块分发、结果汇总与通信开销分析方法。
3. 理解矩阵规模、进程数量与程序运行时间之间的关系。
4. 思考在内存受限和稀疏矩阵场景下，如何进一步设计更高效的矩阵乘法算法。

2　实验原理

设矩阵 $A \in \mathbb{R}^{m\times n}$，$B \in \mathbb{R}^{n\times k}$，则乘积矩阵 $C \in \mathbb{R}^{m\times k}$ 的定义为

$$C_{i,j} = \sum_{p=1}^{n} A_{i,p} B_{p,j}. \tag{1}$$

经典三重循环矩阵乘法的时间复杂度为 $O(mnk)$，对应的浮点运算量可近似为

$$\text{FLOPs} \approx 2mnk. \tag{2}$$

若程序运行时间为 t 秒，则浮点性能可表示为

$$\text{GFLOPS} = \frac{2mnk}{t \times 10^9}. \tag{3}$$

本实验采用 MPI 对矩阵乘法进行并行化。其基本思路如下：

1. 由 0 号进程随机生成矩阵 A 和 B。

2. 将矩阵 A 按行划分为若干连续块，分发到各个进程。
3. 将矩阵 B 广播到所有进程，使每个进程都能访问完整的右矩阵。
4. 各进程独立计算自己负责的 A 子块与 B 的乘积，得到局部结果块。
5. 将局部结果块回收至 0 号进程，拼接成最终矩阵 C。

这种实现的优点是结构清晰、易于实现，适合验证 MPI 并行矩阵乘法的基本性能特征。其主要开销来自两部分：一是矩阵块分发与回收的通信开销，二是各进程本地乘法计算的时间。随着进程数量增加，计算部分通常下降，但通信与同步开销会逐渐上升，因此加速比不会无限线性增长。

3 实验内容

本实验在 `lab/01/src` 中完成，核心程序为 MPI 矩阵乘法实现，编译后生成可执行文件 `matmul_mpi`。程序输入为 m, n, k 三个整数，矩阵元素随机生成，默认输出矩阵 A、B、C 以及计算时间和 GFLOPS。

实验内容包括以下几部分：

1. 实现 MPI 版本的通用矩阵乘法程序。
2. 使用 `mpirun` 在不同进程数下运行程序，观察运行时间变化。
3. 在不同矩阵规模下测试程序性能，分析规模扩大对耗时和吞吐率的影响。
4. 回答实验要求中的讨论题，并总结当前实现的局限性。

实验运行方式如下：

```
cd lab/01/src
make
mpirun --oversubscribe -np 4 ./matmul_mpi 512 512 512 --no-print
python3 benchmark.py \
    --sizes 128,256,512,1024,2048 \
    --processes 1,2,4,8,16 \
    --skip-build \
```

实验结果

表格如下。由于矩阵乘法属于计算密集型任务，当矩阵规模较小时，MPI 通信开销和进程启动开销会比较显著；当规模增大后，计算量增长更快，程序更容易体现并行带来的收益。与此同时，如果进程数继续增加而问题规模不变，每个进程分到的工作会变少，此时通信与同步占比会上升，导致效率下降。

Size	np=1	np=2	np=4	np=8	np=16
128	0.003445 s	0.004889 s	0.003381 s	0.002468 s	0.002539 s
256	0.004088 s	0.003379 s	0.002596 s	0.009158 s	0.003148 s
512	0.023330 s	0.013122 s	0.008196 s	0.009276 s	0.012584 s
1024	0.206837 s	0.127166 s	0.114528 s	0.120935 s	0.203254 s
2048	3.262171 s	2.246921 s	1.327422 s	1.273556 s	1.131457 s

讨论题回答

1. 在内存有限的情况下，如何进行大规模矩阵乘法计算？

当矩阵无法一次性完整装入内存时，应采用分块计算、外存计算或流式计算策略。基本思想是把大矩阵切分成多个较小的子块，每次只加载参与当前计算的块进入内存，计算局部乘积后立即写回结果。这样可以显著降低峰值内存占用。

在并行环境中，常见做法包括：

1. **二维分块**：把 A、B、C 都切成子块，让多个进程协同计算不同块。
2. **分阶段广播**：不一次性广播完整矩阵，而是按列块或行块分批传输。
3. **外存分块**：当内存不足时，将中间块缓存到磁盘，采用多轮 I/O 与计算交替进行。
4. **通信与计算重叠**：在加载下一批数据的同时计算当前批次，减少空等时间。

对于极大规模矩阵，合理的块大小选择非常关键：块太小会导致通信和调度开销过大，块太大又会增加内存占用并降低缓存命中率。实际实现中通常需要结合机器内存容量、缓存层次结构和网络带宽进行权衡。

2. 如何提高大规模稀疏矩阵乘法性能？

稀疏矩阵乘法的关键不在于减少循环层数，而在于尽量避免对零元素进行无效计算。对于稀疏矩阵，若仍使用普通稠密矩阵乘法，会浪费大量时间和内存。因此通常需要采用稀疏存储格式和稀疏算法。

可行的优化方向包括：

1. **采用稀疏存储格式**：如 CSR、CSC、COO 等，只存储非零元素及其索引。
2. **基于非零结构进行计算**：乘法时只遍历非零项，并利用索引快速定位对应元素。
3. **负载均衡**：稀疏矩阵中各行/各块的非零数可能差异很大，MPI 划分时不能只按行数平均分配，而应按非零元数量或预计计算量进行分配。

4. **局部性优化**：对非零结构进行重排或分块，以提高缓存命中率和向量化效率。

5. **通信压缩**：并行时只传输非零块或必要索引，减少网络通信量。

因此，稀疏矩阵乘法的核心优化目标是“跳过零计算、减少无效数据搬运、保持负载均衡”。如果稀疏模式高度不规则，通常还需要结合图划分、超图划分或动态任务调度进一步优化。

结果分析

本实验的实现属于基础的行块并行方案，优点是简单稳定，适合验证 MPI 并行矩阵乘法的基本流程；不足之处在于 B 被完整广播到所有进程，存在一定的内存重复占用和通信开销。对于更大规模问题或更高进程数场景，可以进一步采用二维块分布、SUMMA 类算法或分层通信方案，以降低广播和回收成本。

4 实验总结

通过本实验，我完成了基于 MPI 的并行通用矩阵乘法实现，并理解了并行矩阵乘法的基本设计方法。实验表明，矩阵乘法的性能不仅取决于计算量本身，还受到数据分布方式、通信开销和负载均衡的显著影响。

本实验的主要收获如下：

1. MPI 并行程序需要同时考虑通信和计算，不能只关注局部计算效率。

2. 对于矩阵乘法这类计算密集型任务，矩阵划分方式直接影响并行效率。

3. 在内存受限和稀疏矩阵场景下，算法设计思路需要从“直接乘法”转向“分块、压缩和结构化计算”。

总体而言，本实验帮助我建立了从串行优化过渡到并行优化的思维方式，也为后续进一步学习更高效的并行矩阵乘法算法打下了基础。